

GUIA RESUMEN — Makefile Mastery

Resumen integral para dominar Makefiles en ~20 minutos, sin volver a leer el modulo completo.

0 Flujo de aprendizaje general

7 ARCHIVOS · ~20 MIN

1 Que es Make 3 min	2 Sintaxis 4 min	3 Variables 3 min	4 Makefile real 4 min	5 Creacion 3 min	6 CI/CD 2 min	7 Ejercicios 3 min
---------------------------	------------------------	-------------------------	-----------------------------	------------------------	---------------------	--------------------------

Regla de oro: "Make es la capa de abstraccion que conecta Git, Flutter, Supabase y herramientas de calidad en una interfaz unificada."

Seccion	Que aprendes	Archivo clave
1. Que es Make	Por que Makefile y no scripts sueltos	01-que-es-make.md
2. Sintaxis basica	Targets, prerequisites, recipes, .PHONY, variables	02-sintaxis-basica.md
3. Variables y shell	\$(shell ...), awk, sed, debugging	03-variables-y-shell.md
4. Makefile real	Recorrido linea por linea del Makefile del monorepo	04-analisis-makefile-real.md
5. Creacion	Template limpio + targets cross-module	05-creacion-personalizada.md
6. CI/CD	Make en GitHub Actions	06-make-en-ci.md
7. Ejercicios	Practica: leer, modificar y crear Makefiles	07-ejercicios.md

1 Que es Make (3 min)

FUNDAMENTOS

Make es una herramienta de automatizacion que lee un `Makefile` y ejecuta comandos segun reglas que defines.

Por que Makefile y no scripts sueltos?

Aspecto	Scripts sueltos	Makefile
Archivos	10+ scripts	1 archivo
Dependencias	Manual	Automatico
Documentacion	Separada	make help
Estandar	No	Universal en DevOps

Make como orquestador

Target	Modulo	Que hace
<code>make commit</code>	12-GIT-FLOW	Commitizen (Conventional Commits)
<code>make release</code>	12-GIT-FLOW	standard-version (SemVer)
<code>make branch</code>	12-GIT-FLOW	Conventional Branch
<code>make check</code>	Quality	format + analyze + test

Ejemplo: `make commit` ejecuta Commitizen → Husky valida → Commit creado con formato correcto.

Estructura de una regla

```
target: prerequisites
      recipe
```

Parte	Que es	Ejemplo
target	Nombre del comando	test:
prerequisites	Que debe ejecutarse antes	format analyze
recipe	Que realmente hace (TAB)	flutter test

.PHONY — La regla mas importante

```
.PHONY: test clean setup
```

Todo target que no genere un archivo debe ser `.PHONY`.

Variables

```
# Asignacion simple (= evaluacion perezosa)
PROJECT_NAME = mi-proyecto

# Asignacion inmediata (:= evaluacion al momento)
MOBILE_DIR := apps/mobile

# Asignacion condicional (?= solo si no esta definida)
DEVICE ?= emulator-5554
```

Target por defecto

```
.DEFAULT_GOAL := help
```

Cuando ejecutas solo `make`, ejecuta `help`.

3 Variables y shell (3 min)

\$(shell ...) — Ejecutar comandos

```
# Detectar Flutter (FVM o global)
FLUTTER := $(shell [ -d .fvm ] && echo fvm flutter || echo flutter)

# Detectar dispositivo Android
DEFAULT_ANDROID = $(shell flutter devices 2>/dev/null \
  | awk -F " " '/android/ {gsub(/ /,"",$2); print $2; exit}')

```

Variables automaticas

Variable	Significado
<code>\$@</code>	Nombre del target actual
<code>\$<</code>	Primer prerequisite
<code>\$^</code>	Todos los prerequisites

Debugging

```
# Imprimir variable
$(info FLUTTER = $(FLUTTER))

# Ejecutar sin ejecutar (solo mostrar)
make -n [target]
```

Variables del proyecto

```
MOBILE_DIR := apps/mobile
WEB_DIR := apps/web
SUPABASE_DIR := supabase
```

Colores para output

```
BOLD := $(shell tput bold 2>/dev/null)
GREEN := $(shell tput setaf 2 2>/dev/null)
YELLOW := $(shell tput setaf 3 2>/dev/null)
CYAN := $(shell tput setaf 6 2>/dev/null)
RESET := $(shell tput sgr0 2>/dev/null)
```

Conventional Commits

```
.PHONY: commit
commit:
    @npm run commit
```

Flujo: Commitizen → Husky pre-commit → Husky commit-msg → Commit creado

Target make release

```
.PHONY: release
release:
    @npm run release
    @git push --follow-tags
```

Flujo: standard-version detecta cambio → incrementa version → genera CHANGELOG → crea tag → push

Mapa de dependencias

```
help          → (autonomo)
setup         → deps → mocks
dev           → check → run
check         → format → analyze → test
validate      → check ✓
build-apk     → env-prod-check ✓
```

Template minimo

```
PROJECT_NAME := mi-proyecto
MOBILE_DIR := .
.DEFAULT_GOAL := help

.PHONY: help
help:
    @echo "$(PROJECT_NAME) - Comandos disponibles"
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
    | sort | awk 'BEGIN {FS = ":.*?## "}; {printf " %-25s %s\n", $$1, $$2}'

.PHONY: setup
setup: deps mocks ## Instalar dependencias y generar codigo

.PHONY: check
check: format analyze test ## Verificacion completa
```

Targets cross-module

```
.PHONY: commit
commit: ## Crear commit (Conventional Commits)
    @npm run commit

.PHONY: release
release: ## Crear release (SemVer + CHANGELOG)
    @npm run release
    @git push --follow-tags

.PHONY: branch
branch: ## Crear rama feature (Conventional Branch)
    @read -p "Nombre de la feature: " name; \
    git checkout develop && git pull && \
    git checkout -b feature/$$name

.PHONY: hotfix
hotfix: ## Crear rama hotfix
    @read -p "Descripcion del fix: " name; \
    git checkout main && git pull && \
    git checkout -b hotfix/$$name
```

Buenas practicas

- ✓ `cd $(DIR) && comando` — no afecta el shell padre
- ✓ `@echo "mensaje"` — feedback visual sin ruido
- ✓ `command -v herramienta` — verificar existencia
- ✓ `.PHONY` en todos los targets
- ✓ `##` para descripciones — auto-documentacion

Por que usar Make en CI?

```
# SIN Make: acciones repetitivas
steps:
  - run: cd apps/mobile && flutter pub get
  - run: cd apps/mobile && dart format .
  - run: cd apps/mobile && flutter analyze
  - run: cd apps/mobile && flutter test

# CON Make: un solo comando
steps:
  - run: make check
```

Comandos en CI/CD

Comando	En CI hace
<code>make setup</code>	pub get + build_runner
<code>make check</code>	format + analyze + test
<code>make coverage</code>	Tests con cobertura
<code>make build-apk</code>	Compila APK release

Ventaja: Local y CI ejecutan exactamente lo mismo. Cambiar la logica en un solo lugar.

A Anexo · Indice del modulo (para profundizar)

Necesitas	Archivo
Que es Make y por que lo usamos	01-que-es-make.md
Sintaxis basica de Makefile	02-sintaxis-basica.md
Variables avanzadas y funciones shell	03-variables-y-shell.md
Analisis del Makefile real del proyecto	04-analisis-makefile-real.md
Creacion personalizada + targets cross-module	05-creacion-personalizada.md
Make en GitHub Actions	06-make-en-ci.md
Ejercicios de practica	07-ejercicios.md

C Conclusion

Make es la **capa de abstraccion** que conecta todas las herramientas del proyecto:

- ✓ Make como orquestador entre modulos (Git, Flutter, Supabase)
- ✓ Sintaxis basica: targets, prerequisites, recipes
- ✓ Variables y funciones shell para automatizacion
- ✓ Targets cross-module: commit, release, branch, hotfix
- ✓ Integracion con GitHub Actions (CI/CD)
- ✓ Buenas practicas y debugging

Tiempo de lectura del resumen: ~20 min · Make es la herramienta que conecta todo en tu proyecto Flutter + Supabase.